SOURCE-LEVEL GUIDE • EGRAPHS-GOOD/EGGLOG

egglog 源码导读

egglog 把等式饱和和 Datalog 合成一门语言。这份导读讲清一条 rewrite 怎么一路变成 Free Join 查询，union-find 和 rebuild 怎么配合，半朴素求值怎么省掉重复工作，以及想贡献或嵌入它时要知道什么。

egraphs-good/egglog @ 6c3a0c9

怎么读这份导读

这份导读写给有编译器或数据库背景、第一次读 egglog 源码的工程师。读完你应该能说清：一条 `rewrite` 怎么变成数据库查询；为什么 e-graph 可以看成一个带 union-find 的数据库；rebuild 和半朴素求值各解决什么；新 sort、新原语、新调度器该写在哪。

全文顺序：

1. **全景**：问题、目标、三层 crate 结构。
2. **前端**：解析、去糖、类型检查、Core IR。
3. **桥接层**：把 e-graph 语义编码到关系上。
4. **union-find 与 rebuild**：等价关系怎么维护。
5. **半朴素求值**：只处理新数据。
6. **查询引擎**：core-relations、树分解、Free Join。
7. **提取与证明**。
8. **调度与并行**。
9. **端到端**：一个 rewrite 的完整一轮。
10. **许可证、贡献与定制、源码索引**。

> **要点 · 固定版本**
>
> `egraphs-good/egglog` @ `6c3a0c9`（2026-11-01），版本 `3.0.0`。我没有运行过这份代码，内容来自阅读源码和注释。

全景：问题、目标、限制

等式饱和和 Datalog 为什么要合在一起

等式饱和用 e-graph 同时表示大量等价程序：不断应用重写规则，但不删除旧项，最后再提取最好的一个。这避开了重写顺序问题。**Datalog** 擅长基于规则的程序分析，增量求值和查询优化都很成熟。

两者各有缺口：

环节	传统 e-graph 的问题	Datalog 的问题	egglog 的回答
模式匹配	逐个 e-node 搜索，慢	用关系 join，快	把 e-matching 变成关系查询
增量	每轮重新匹配全部	半朴素求值	用时间戳实现半朴素
等价	内建	没有，要手写规则	内建 union-find + 重建
分析	需要定制	规则就是分析	函数、lattice merge

README 称它为“下一代等式饱和引擎”。理论基础是论文 *Better Together: Unifying Datalog and Equality Saturation*，见 `CITATION.bib`。

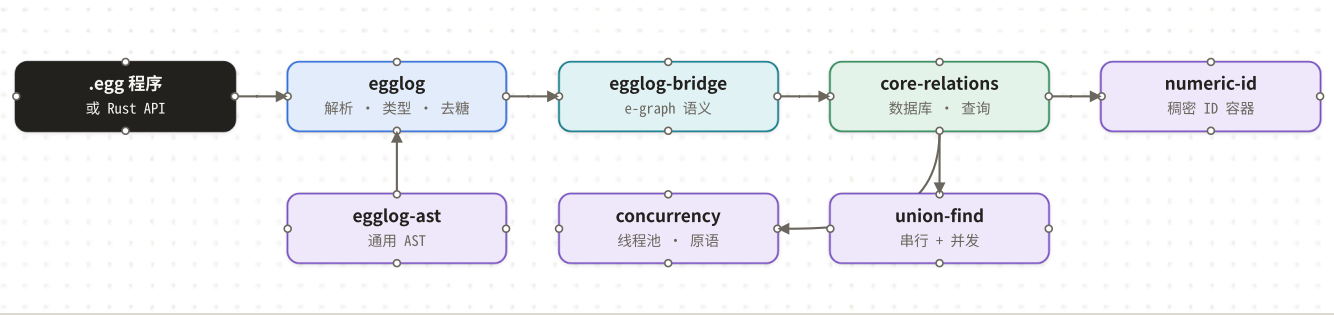
项目的一句话定义来自 `CLAUDE.md`：**可以把 egglog 看成 Datalog 加上一个 union-find。**

要求 / 目标 / 限制

项目	内容
核心目标	用一个 Datalog 风格的语言同时做程序优化和程序分析
功能要求	规则、函数、sort、调度、提取、证明；可作 Rust 库、CLI、WASM 使用
规模压力	数据库规模随饱和爆炸增长，查询计划必须好
外部限制	要支持单线程和并行；要能编译到 WASM；Rust 2024 edition
关键手段	时间戳半朴素、Free Join、树分解、原地 union-find 表、term encoding
许可证	MIT

crate 分层

从语言到数据库的四层



分工见 `Project structure`。注意 `bridge 的文件头` 的说法：它主要做“去糖”，不实现 join 或 union-find 算法。workspace 成员见 `Cargo.toml`。

crate	职责	入口
`egglog`	语言前端、类型检查、证明、提取、调度	`src/lib.rs`
`egglog-bridge`	半朴素、默认值、merge 函数	`lib.rs`
`core-relations`	表、索引、Free Join、动作 VM	`lib.rs`
`union-find`	串行和并发 union-find	`lib.rs`
`concurrency`	线程池和并发结构	`lib.rs`

前端：从文本到 Core IR

主要路径

`parse_and_run_program` 解析后交给 `run_program`，再进入 `process_program_internal`。它**逐条命令**处理：

▶ **一条命令的处理流程** 1 / 5



1. 宏展开

先做用户提供的宏展开。它按命令逐条进行，所以后面的宏能依赖前面命令产生的类型信息（代码）。

2. 去糖

`desugar_command` 把 `rewrite`、`birewrite`、`datatype`、`relation` 等转成更小的核心命令。比如 `desugar_rewrite` 把重写变成一条规则。

3. 类型检查

`typechecking.rs` 用 `constraint.rs` 的约束求解来确定类型，产出解析后的命令。

4. 证明编码

只有开启 term encoding 时才走这一步：`resolve_command` 去掉全局变量，再由 `ProofInstrumentor` 重写程序，然后再次去糖、类型检查。

5. 执行

`run_command` 按命令类型分发。遇到规则就编译，遇到 `run` 就运行。

错误语义有个明确约定：`run_program` 的文档说 egglog 错误**不是事务性的**，失败前完成的效果都保留；规则动作出错后，一次成功的恢复 rebuild 让数据库重新规范且可用（文档注释）。

Core IR

`core.rs` 文件头讲清了这一层的定位：**规则体是合取查询，规则头是类似 SSA 的动作序列**。类型检查和规范化都在这个格式上做，大部分编译期优化也应该放这里。

抽象	作用
`GenericAtom`	查询里的一个原子：`f(x, y) = z`
`Query`	原子的合取
`GenericCoreAction`	动作：set、union 等
`GenericCoreRule`	查询 + 动作
`ResolvedCall`	解析到函数或具体原语

函数即表

egglog 的核心抽象：**每个函数或构造器都是一张表**，输入列是键，输出列是值。`(Add x y)` 就是表 `Add` 里的一行 `(x, y) → e-class id`。e-node 不再是专门的数据结构，只是表里的行。

桥接层：e-graph 语义

职责

`egglog_bridge::EGraph` 在 core-relations 上实现三件事：半朴素、默认值、merge 函数。它不实现算法，只做编码。

一张函数表的列比用户看到的多：键列、值列，再加时间戳等元信息（新旧行靠它区分）。规则由 `rule.rs` 的 builder 构建。文件头说得很准：**egglog 规则最终只是一组 core-relations 规则，用时间戳范围作为约束来参数化。**

`add_rule` 负责连接两层：

1. 把解析后的规则规范化成 Core IR。
2. 决定是否半朴素。`:naive` 规则退出半朴素，这时原语可以读写数据库。
3. 用 `BackendRule` 把查询和动作翻译成 bridge 的 builder 调用。

> **注意 · 容易误解的地方**
>
> 半朴素不只是优化，它还影响语义：只有 `:naive` 或 `:unsafe-seminaive` 规则才能用读数据库的原语。看到 `requires_read_context` 时要想到这一点。

union-find 与 rebuild

问题

当 `union(a, b)` 发生后，所有引用 `b` 的行都过期了：`f(b) = x` 和 `f(a) = y` 现在键相同，x 和 y 必须合并，这可能触发新的 union。这叫同余闭包。

union-find 本身

`union-find` 用 **union by min id**：小 ID 当根。文件头明说这不保证文献里的渐进复杂度，这是个启发式选择，好处是减少同余闭包中被扰动的 ID 数量，实现也简单。

并发版本的安全性论证写在注释里：成功的 union 把根指向更小的 ID，路径分裂也只把父节点改成更小的祖先，所以链接单调递减，不会有 ABA，不会成环（`UnionFind` 附近）。

union-find 自身也是一张表：`uf/mod.rs` 是“以 union-find 为后端的表”，所以 union 也是往表里写一行。

> **类比 · 公司合并后的员工号**
>
> - **员工号** = e-class id。
> - **两家公司合并** = `union`：两个号从此算同一个人，较小的号保留。
> - **改名登记册** = union-find：查旧号能找到现行号。
> - **全公司的表格** = 函数表：里面还写着旧号，所以要逐张更新。
> - **重复的人事记录要合并** = 同余：更新后发现两行键相同，薪资字段要走 merge 规则，可能又发现两个人其实是一个人。
>
> 一直做到没有新发现为止，这就是 rebuild 循环。

rebuild

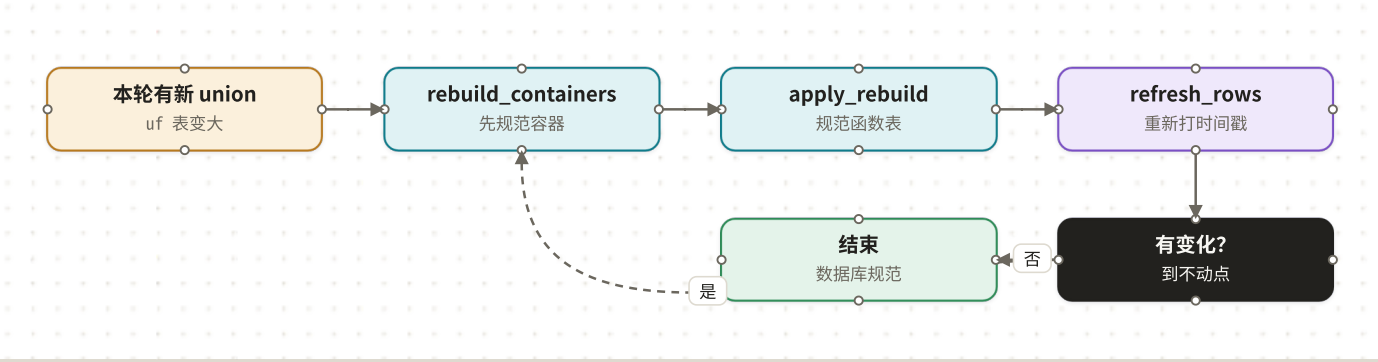
`run_rules_inner` 有个小优化：如果这轮 union-find 表大小没变，就跳过 rebuild，只推进时间戳。

`rebuild` 分三种情况：

路径	条件
原生表级 rebuild	union-find 表提供 `rebuilder`
并行 rebuild	线程数大于 1
串行迭代	其他情况

原生路径的循环值得细看：**必须先 rebuild 容器，再 rebuild 表**。这段注释用一个 `Vec` 例子解释了原因：如果先 rebuild 表，`:no-merge` 函数可能因为两个 `Vec` 尚未被识别为相等而报错。

rebuild 循环



对应 `rebuild 的循环`。`refresh_rows` 处理一种细节：容器 rebuild 可能让父行变得可匹配，但存的 id 没变，所以要重新打时间戳，否则半朴素看不到。

最新的 CHANGELOG 还提到增量 rebuild 只在反向索引足够“热”时才启用（`CHANGELOG`）。

半朴素求值

问题

一轮规则跑完，下一轮绝大多数匹配都是旧的。全量重新匹配浪费极大。

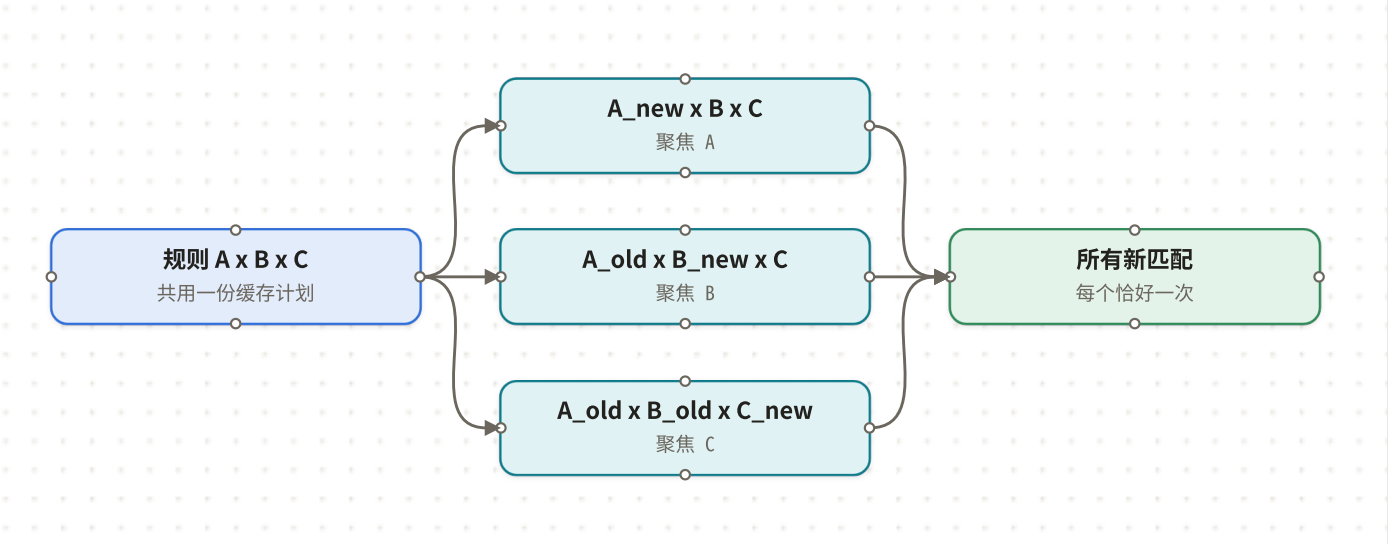
原理

每行带时间戳。每条规则记住 `last_run_at`。对 N 个原子的规则，生成 N 个查询变体：第 i 个查询要求第 i 个原子只匹配**新行**，它前面的原子匹配旧行，后面的匹配全部。这保证每个新匹配恰好被找到一次。

`rule.rs` 的注释正是这么说的：“For N atoms, we create N queries for seminaive evaluation”，并且复用缓存的计划，只改时间戳约束。

> **类比 · 只看未读邮件**
>
> - **收件箱** = 函数表。
> - **已读标记** = `last_run_at`。
> - **规则** = 过滤器：“A 发的并且有 B 附件”。
>
> 全量匹配等于每天把全部邮件再筛一遍。半朴素只找**新邮件**参与的组合：新的 A 配任意 B，或旧的 A 配新的 B。任何完全由旧邮件组成的组合，昨天已经处理过了。

三个原子的规则生成三个变体



公式直接来自 `add_rules_from_cached` 的注释。第一次运行时 `mid_ts` 为 0，旧集合为空，所以聚焦后面原子的变体直接跳过。

`run_rules_impl` 把所有规则放进一个 rule set 一起跑，每条规则的 `last_run_at` 更新为当前时间戳。第一次运行时计划才构建并缓存。

查询引擎：core-relations

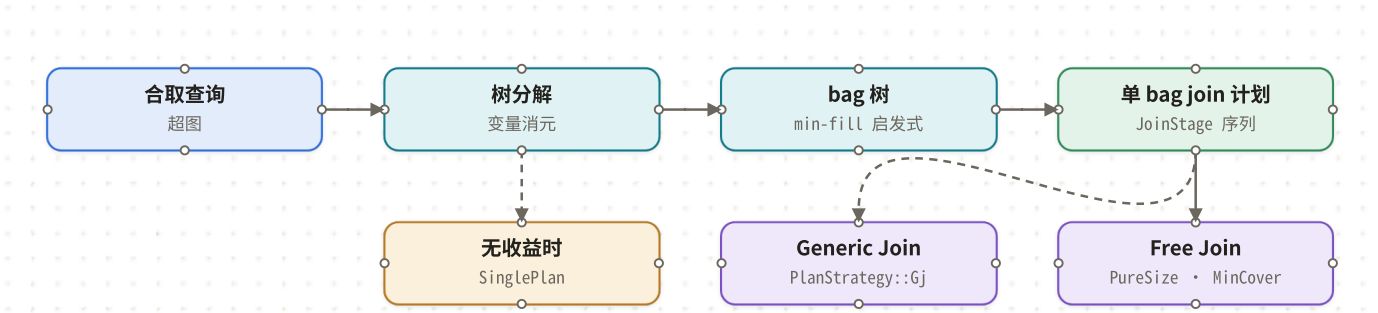
职责

这是一个小型内存数据库，提供表、索引、查询计划和一个小的动作 VM。`Database` 是入口，`Table` 是表的 trait，所以 union-find 表和普通排序表可以共存（`table/`、`uf/`）。

查询规划

`plan.rs` 的文件头是全项目最好的文档之一，分两个阶段：

查询计划的两个阶段



`:no-decomp` 可以跳过分解（`add_rule`）。策略枚举是 `PlanStrategy`。

概念	含义
树分解	把查询超图拆成一棵重叠子查询树，像 Yannakakis 算法
变量消元	每次选一个变量，产生一个 bag。`next_var_to_eliminate` 用 min-fill 启发式
消息变量	bag 的结果按它和父节点共享的变量做键，父节点用它剪枝
Generic Join	每步选一个变量，与所有相关列求交集，最坏情况最优
Free Join	每步选一个 cover 子原子去探测其他原子。可以退化成哈希 join 或接近 Generic Join

执行在 `execute.rs`，是整个引擎最大的文件之一。`packed_trie.rs` 提供紧凑索引。CHANGELOG 里最近的工作集中在这里：同一 rule set 运行内共享 packed join 索引。

动作

规则头编译成 `action` 里的小型 VM 程序，这与 `core.rs 文件头` 的说法一致。写入进入缓冲，之后 `merge_all` 合并。

提取与证明

提取

饱和之后要从 e-class 选一个最便宜的项。`extract.rs` 把代价模型分成两个 trait：

Trait	用途
`TreeCostModel`	树代价：节点注解加子节点代价折叠
`DagCostModel`	DAG 代价：共享的子项只算一次
`MonoidCost`	DAG 需要的结合、交换组合运算

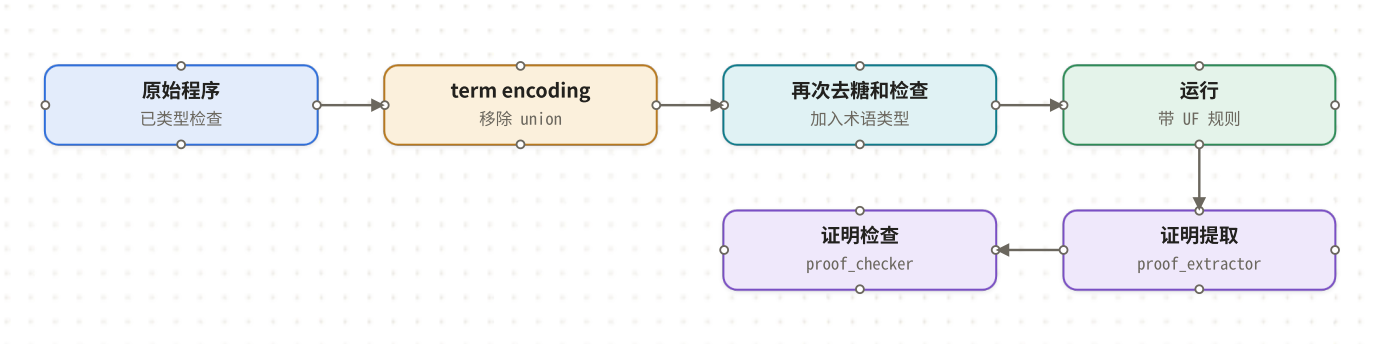
这是刚发布的破坏性变更（`CHANGELOG`）。自定义代价要满足的代数律文档里写明了，Rust 无法强制。

证明

这是比较新的部分。`proof_encoding.md` 讲清了核心思路：**term encoding 把程序里所有 `union` 调用去掉**。每个 sort 显式添加 union-find 结构，用规则维护，这样全部等式推理都显式出现，可以被加上证明跟踪。

这是**程序到程序的变换**：普通 egglog 程序被重写成另一个 egglog 程序。

证明模式的结构



开启方式是 `EGraph::new_with_proofs`，代码在 `src/proofs/`。`proof_encoding_helpers` 还导出了判断程序是否支持证明的函数，不是所有程序都支持。

调度与并行

调度

默认每个 ruleset 跑一轮并应用全部匹配。`Scheduler` trait 提供两个钩子：

方法	作用
`can_stop`	无数据库变化时能否认为饱和
`filter_matches`	筛选一条规则的匹配

这是控制指数爆炸的关键手段。一条规则选中的匹配会立即应用，然后才调用下一条规则的 `filter_matches`（`CHANGELOG`）。

用户也可以在语言层写 schedule：`run_schedule` 执行组合的 `repeat`、`seq`、`saturate`。

并行

`-j N` 指定线程数，默认 1。README 提醒：这个支持比较新，多数用户单线程运行，基准也只测单线程，但不要加入粗粒度锁（`Parallelism`）。每个 `EGraph` 有自己的作用域线程池，实现在 `concurrency`。CLAUDE.md 也写明 core-relations 默认串行。

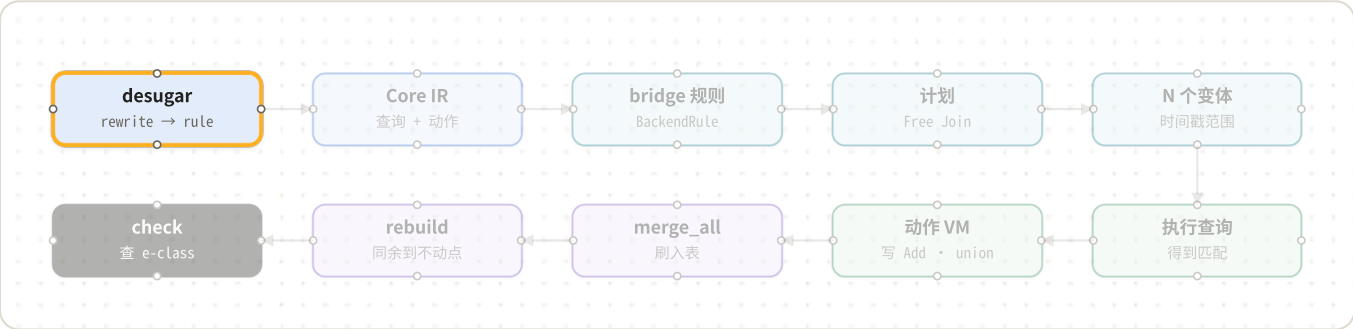
端到端：一条 rewrite

看一个程序：

```
(datatype Math (Num i64) (Add Math Math))
(rewrite (Add a b) (Add b a))
(let e (Add (Num 1) (Num 2)))
(run 1)
(check (= e (Add (Num 2) (Num 1))))
```

这是我写的示意，用来串流程，没有实际运行过。

▶ 一条 rewrite 的一轮 1 / 10



1. 去糖

`rewrite` 变成规则：匹配 `(Add a b)` 得到 `?e`，动作是构造 `(Add b a)` 并和 `?e` union。

2. Core IR

规范化后，查询是单原子 `Add(a, b) = e`，动作是一串 SSA 式的操作。

3. 翻译

`add_rule` 通过 `BackendRule` 翻译，用 `seminaive = true` 建 bridge 规则。

4. 计划

第一次运行时 `build_cached_plan` 构建并缓存计划。单原子很简单，多原子才用到分解和 Free Join。

5. 时间戳

只有一个原子，所以只有一个变体：只看 `last_run_at` 之后的新行。

6. 查询

`Database` 的 `run_rule_set` 执行，找到 `(Add (Num 1) (Num 2))`。

7. 动作

VM 查找或创建 `(Add (Num 2) (Num 1))`，再写一条 union。

8. 合并

缓冲写入刷入表。

9. rebuild

uf 表变大了，所以 rebuild：规范函数表，处理同余。

10. 检查

`check` 把事实编译成查询，确认两边在同一 e-class。

开源许可证

部分	许可证	证据
全部 crate	MIT，Copyright 2025 egraphs-good	`LICENSE`、`workspace 声明`
依赖	见各 crate	`workspace.dependencies`

场景	允许吗	要做什么
自用、嵌入产品	允许	保留 LICENSE
提 PR	允许	未看到 CLA
公开 fork	允许	保留声明
发 crate	允许	换名字
对外提供服务	允许	无网络条款

历史：GitHub API 显示 `LICENSE` 只有 2025-01-18 一次提交记录。学术使用请按 `CITATION.bib` 引用。

> **不是法律意见**
>
> 这是工程师读代码时的理解。

贡献与定制

开发环境

```
git clone https://github.com/egraphs-good/egglog
cd egglog
cargo run --release -- tests/fibonacci-demand.egg
make test
make fixnits
```

项目规则

来自 `CLAUDE.md`：

- 结束前跑 `make fixnits` 或 `make nits`。
- 测试总是用 `--release`，或 `make test`。
- 性能敏感的改动，以 `script/bench.py` 为准。
- 文档保持简洁，避免重复。
- 重大改动更新 `CHANGELOG.md`。
- PR 有 git 大小预算检查，防止误提交生成数据（`Git size budget`）。
- 不要悲观化并行性能，比如加粗粒度锁。

测试是快照风格：`tests/` 下有约 100 个 `.egg` 程序，`make test` 用 `cargo insta`。

适合入手的方向

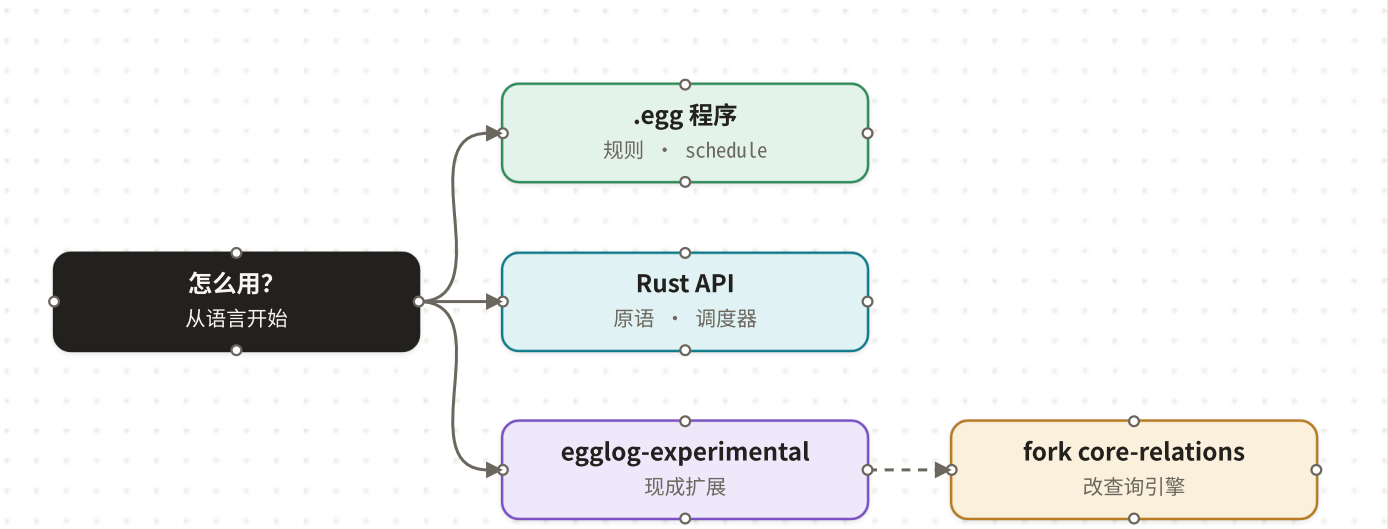
- 新增 `tests/*.egg` 回归用例。
- 补 sort 的原语，模板在 `src/sort/`。
- 改善错误信息。
- 文档，比如公开 API 的 rustdoc。

扩展点

想加什么	位置	说明
新 sort	实现 `Sort`	参考 `vec.rs`
新原语	`Primitive` + `add_primitive!`	分 Pure、Read、Write、Full 四种上下文
新命令	`UserDefinedCommand`	`add_command` 注册
命令宏	`CommandMacro`	可以看到类型信息
调度器	`Scheduler`	控制匹配
代价模型	`extract.rs`	遵守代数律

定制路线

想在 egglog 上做点什么



README 推荐需要更多功能时用 egglog-experimental，Python 绑定在 egglog-python。

仓库里没有的东西

- **语言绑定和实验插件**：分别在 `egglog-python` 和 `egglog-experimental`。
- **教程**：在 egglog-tutorial。
- **nightly 基准结果**：在 UW 的机器上跑，脚本在仓库里（`Makefile`）。

附录：源码索引

主题	链接
入口	`src/lib.rs`
CLI	`cli.rs`
AST 与解析	`ast/mod.rs`、`parse.rs`
去糖	`desugar.rs`
类型	`typechecking.rs`、`constraint.rs`
Core IR	`core.rs`
调度	`scheduler.rs`
提取	`extract.rs`
证明	`proof_encoding.rs`、`proof_checker.rs`
桥接	`lib.rs`、`rule.rs`
数据库	`free_join/mod.rs`
计划	`plan.rs`
执行	`execute.rs`
表	`table/mod.rs`
union-find	`union-find`、`uf/mod.rs`
并发	`concurrency`
WASM	`wasm-example`